

Competitor Price Tracker for Car Dealers

Watches competitor dealer websites every day: new stock, price cuts, sold cars, days on site, and where your prices sit against the market

IN ONE SENTENCE

A polite daily scraper plus dashboard that tells a dealer what competitors added, cut and sold, and whether their own cars are priced right.

BUILT WITH

Node 22, TypeScript, React dashboard; production: Fastify 5, PostgreSQL, SMTP, Docker

What this guide covers

- 1 What the project is
- 2 The business problem it solves
- 3 The client's requirements
- 4 How the solution is designed
- 5 How to build it from scratch, step by step
- 6 Testing, deployment and known limits

Contents

1. What this project is	3
2. The business problem	4
3. Client requirements	5
4. How the solution is designed	7
5. Build it from scratch, step by step	9
6. Testing, deployment and known limits	13

1. What this project is

Used-car pricing is local and fast-moving. Dealers check competitors' websites by hand to see what they have in stock and at what price. This project automates that: every day it reads competitor dealers' public inventory pages, records each car, and turns the daily snapshots into a **change feed** (new listings, price cuts and rises, sold cars, relists), **days on site** per car and competitor, and a **pricing position** for each of your own cars against comparable listings.

The repository has **two versions**: the root folder is a command-line scraper with a GitHub Action and a static dashboard; `dealer-price-tracker-production-grade` is a server with PostgreSQL, logins, a built-in daily schedule, a "Test first page" button, email digests and protection against the scraper being abused to reach internal networks.

At a glance	
Type	Web scraper + change detection + dashboard
Users	Dealer principal, used-car buyer/pricing manager, sales team (viewer)
Stack	Node 22, TypeScript, React dashboard; production: Fastify 5, PostgreSQL, SMTP, Docker
Extraction	schema.org JSON-LD (Car / Vehicle / Product, ItemList, @graph) or CSS selectors per site

Who uses it

User	What they do with it
Pricing / used-car manager	Reads price cuts and sold cars daily; reprices own stock against the market median.
Dealer principal	Watches competitors' stock levels and how fast they sell.
Admin	Adds competitors, sets the schedule and digest recipients, uploads own stock, manages users.
Viewer (sales)	Reads the dashboard and exports.

Two versions in this repository

	Demo version (root folder)	Production version (....production-grade folder)
Data	JSON files in the repository	PostgreSQL: competitors, snapshots, runs, stock, activity
Who sees it	Anyone with the dashboard URL	Sign-in required; admin and viewer roles
Adding competitors	Edit <code>config/competitors.json</code>	A form with a "Test first page" button
Scheduling	GitHub Action commits results	Built-in daily schedule, "Scrape now", one scrape at a time, crashed runs closed
Alerts	—	Daily digest email: price cuts, new and sold cars, your cars above market, problems
Your stock	CSV in the repo	Upload / download your DMS CSV (Excel UTF-8 supported)
Safety	—	Refuses private, loopback and cloud-metadata addresses, including via DNS and redirects

2. The business problem

Dealers who price by feel either leave money on the table or let cars sit. Manually checking five competitor websites every morning takes an hour and misses things: a competitor cutting prices on the same model, a car that sold in four days, or the fact that your car is 12% above the local market.

The client wants **daily, automatic, polite** competitor tracking, with a dashboard and exports, that never mistakes a broken website for a sell-out.

Pain point today	What it costs the business	How this project solves it
Checking competitor websites by hand	An hour a day; things missed	Daily automatic scrape of every competitor
No idea how fast competitors sell	Wrong buying decisions	Days on site per car and median per competitor
Own cars mispriced	Slow stock or lost margin	Each of your cars vs comparable listings (same make/model, ± 1 year, ± 25 k miles): median and % above/below
A broken page looks like everything sold	False alarms, bad decisions	Empty or suddenly 60%+ smaller scrapes are skipped; failed pages never mark cars sold
Scraping that gets you blocked	Lost data, legal risk	Obeys robots.txt and Crawl-delay, waits between requests, identifies itself

Value to the business

- Faster, evidence-based repricing.
- Early warning of competitor moves.
- Exports for Excel and Power BI.

3. Client requirements

The brief below is written the way a client would hand it to a developer: what the system must do (functional requirements), how well it must do it (non-functional requirements), and what was explicitly left out. Every item below is implemented in the repository; section 6 explains how the project is tested.

Functional requirements

ID	Area	Requirement
FR-01	Scraping	For each competitor: read robots.txt and never fetch disallowed pages; honour Crawl-delay; wait between requests (default 3 s); follow "next page" links with a loop guard and page cap; retry server errors with back-off.
FR-02	Scraping	Extract vehicles from schema.org JSON-LD, or from CSS selectors configured per site (card, title, price, optional link, mileage, VIN, stock number; <code>selector@attr</code> for attributes).
FR-03	Scraping	Normalise prices (\$23,995, PKR 4,250,000, £12.5k), mileage, VIN, year, make and model; strip tracking parameters from URLs; identify each car by VIN, else URL.
FR-04	Change detection	Replay daily snapshots into a change feed: new, price cut, price rise, removed (sold), relisted.
FR-05	Change detection	Skip scrapes that return nothing or 60%+ fewer cars; when any page failed, record new cars and price changes but don't mark missing cars sold; cars seen on the first scrape aren't "new" and their age shows as "N+ days".
FR-06	Dashboard & exports	Dashboard: change feed, stock and median days on site per competitor, your pricing position, charts; phone and dark mode.
FR-07	Dashboard & exports	CSV exports: listings, events, daily (Excel / Power BI).
FR-08	Production additions	Competitor form with Test first page; daily schedule at a chosen time; Scrape now; run history.
FR-09	Production additions	Daily digest email; stock upload/download; admin and viewer logins; activity log.

Non-functional requirements

ID	Quality	Requirement
NFR-01	Good citizenship	robots.txt and Crawl-delay obeyed; at least 1 s between pages (default 3); descriptive User-Agent; one scrape a day; public listing pages only.
NFR-02	Security (production)	Passwords hashed with bcrypt; accounts lock after 5 failures; only hashes of session tokens are stored; sessions expire.
NFR-03	Security (production)	Every change needs a CSRF token and a same-site Origin; strict security headers (Content-Security-Policy, HSTS behind https).
NFR-04	Security (production)	Every input validated on the server (zod); all SQL parameterised; rate limits on the API and tighter ones on login.
NFR-05	Security (production)	An activity (audit) log of logins and every business action, viewable by admins.
NFR-06	Security (production)	The scraper only fetches public internet addresses: every DNS answer and redirect hop is checked (no loopback, private, link-local/cloud-metadata or mapped IPv6); http/https only; at most 5 redirects; page size cap.

ID	Quality	Requirement
NFR-07	Reliability (production)	One scrape at a time even with several app instances; the daily schedule runs once however many instances ask; a run left by a crashed server is closed as failed; a re-scrape that finds nothing never wipes a good earlier result.
NFR-08	Operations (production)	Runs as one Docker image next to PostgreSQL; health checks <code>/healthz</code> and <code>/readyz</code> ; database migrations run on start-up safely with several instances.
NFR-09	Operations (production)	Structured JSON logs with a request id (returned in the <code>x-request-id</code> header); cookies and tokens are redacted.

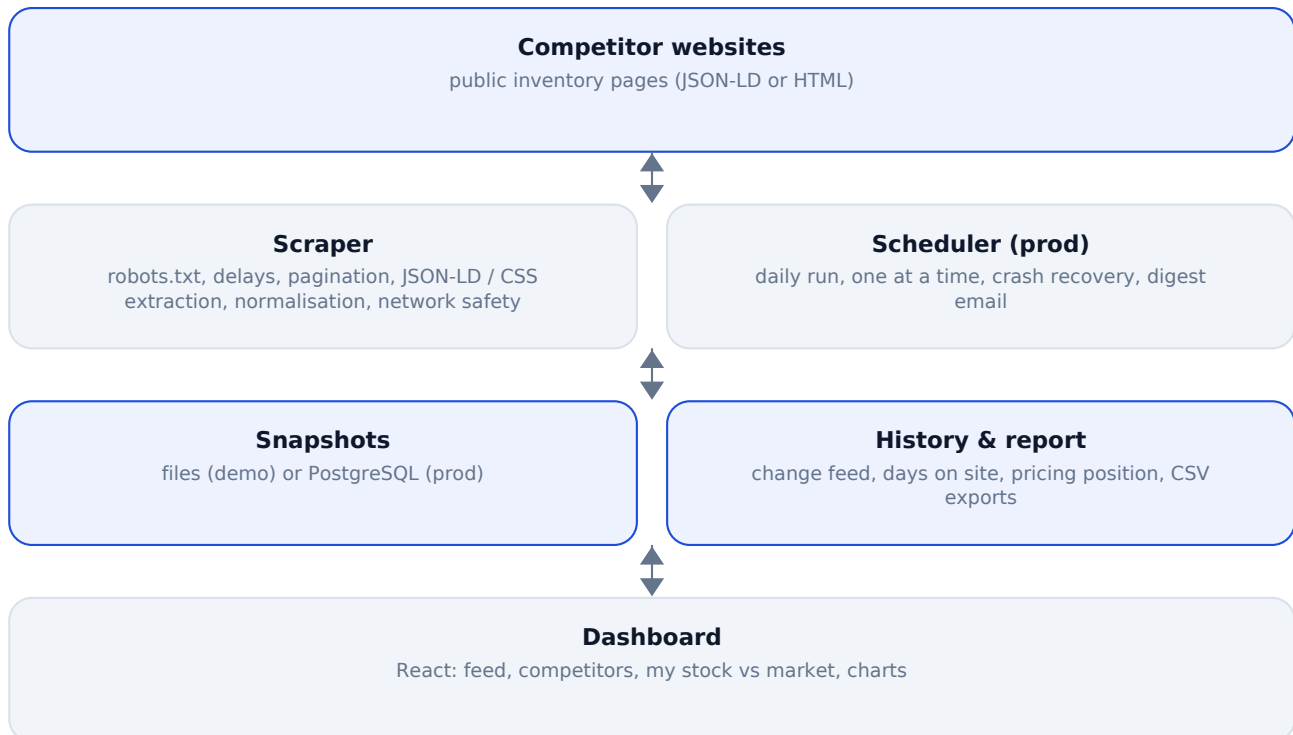
Out of scope for this release

- Sites that build their inventory with JavaScript only (need a headless browser).
- Currency conversion (all competitors should list in your currency).

4. How the solution is designed

A pipeline: scrape → snapshot per competitor per day → replay snapshots into history → report.
The replay design means the analysis can be rebuilt at any time from raw snapshots, and bugs in change detection never corrupt the stored data.

Architecture



Technology choices

Part	Choice	Why
Scraper	Node 22 + TypeScript (fetch, a small HTML/JSON-LD parser)	No browser needed for most dealer sites, which publish JSON-LD.
Dashboard	React + Vite	Static in the demo (GitHub Pages); served by the API in production.
Server (prod)	Fastify 5 + PostgreSQL + SMTP	Logins, scheduling with database locks, digest emails.
Tests	Vitest, Playwright, simulated dealer sites	The scraper is checked car by car against three simulated site formats.

Data model

Table / file	What it holds
competitors	Name, inventory URL, mode (auto JSON-LD or CSS selectors), selectors, next-page selector, enabled.
scrape_runs	Each run: start, finish, status, pages, cars found, errors.
snapshots	Cars seen per competitor per day (VIN/URL id, price, mileage, year, make, model, link).
own_stock	Your cars: stock number, year, make, model, mileage, price.

Table / file	What it holds
report / settings / users / audit_log	Cached dashboard report; schedule, digest recipients, history length; logins; activity log (production).

Key business rules

- **Comparable listing:** same make and model, year within ± 1 , mileage within $\pm 25,000$.
- **Sold** = missing from a complete (no failed pages) scrape after being present.
- **Suspicious scrape** = zero cars, or 60%+ fewer than the last good scrape → skipped.
- **Days on site** counted from first seen; cars present on the very first scrape show "N+ days".

Folder layout

```
Root: src/scrapper (robots, extract, scrape) · src/core (history, normalize, csv) · src/cli (
  scrape, report, demo)
      src/demo (simulated dealer sites) · src/web (dashboard) · config/ · data/
dealer-price-tracker-production-grade/: shared/ · server/ · web/ · docs/ · Dockerfile
```

5. Build it from scratch, step by step

Part A (steps 1–5) builds the scraper and history logic against simulated sites (no internet, reproducible tests) and the dashboard; Part B (steps 6–13) turns it into the production service.

Before you start

- Node.js 22+; PostgreSQL and Docker for Part B.
- Browser developer tools skills (inspecting a car card to write CSS selectors).

Step 1. Part A - Simulated dealer sites

- 1.1 Build three fake dealer websites in `src/demo`: a JSON-LD ItemList with pagination, plain HTML cards with pagination, and a JSON-LD `@graph` page with a broken JSON block; one site goes "down" on one day.
- 1.2 Simulate a market over 60 days (new cars, price cuts, sales) to generate realistic history.

Done when: The simulated sites serve pages locally.

Step 2. Part A - Polite fetching

- 2.1 `robots.ts`: parse robots.txt (user-agent groups, Allow/Disallow, Crawl-delay) and check each URL.
- 2.2 `scrape.ts`: queue pages, wait between requests, follow next-page links with a loop guard and cap, retry 5xx with back-off, identify with a descriptive User-Agent.

Done when: Tests prove disallowed pages are never fetched.

Step 3. Part A - Extraction and normalisation

- 3.1 `extract.ts`: find JSON-LD blocks, walk `@graph` and ItemList, accept Car/Vehicle/Product; fall back to configured CSS selectors.
- 3.2 `normalize.ts`: prices in many formats, mileage, VIN, year/make/model, URL cleanup, stable car id.

Done when: Scraper tests match every car, price and VIN on all three simulated formats.

Step 4. Part A - History and change detection

- 4.1 `history.ts`: replay snapshots in date order into events and per-car state; apply the skip and failed-page rules; compute days on site and your pricing position.
- 4.2 `report` CLI writes `public/data/history.json` and CSV exports.

Done when: The simulated down day is skipped rather than marking cars sold.

Step 5. Part A - Dashboard and automation

- 5.1 React dashboard reading `history.json`: feed, competitors, my stock vs market, phone and dark mode.
- 5.2 `npm run demo` runs the real scraper against the simulated sites for 60 days; a GitHub Action runs `npm run scrape` daily when `config/competitors.json` exists.

```
npm install
npm run demo      # 60 simulated days
npm run dev       # open the dashboard
```

Done when: Unit/integration tests and the browser smoke test pass.

Step 6. Part B · Set up the workspace

One repository with a server, a web app and shared code used by both.

- 6.1 Create an npm workspace with `server` and `web` packages and a `shared` folder that both import (types, zod schemas and business rules).

```
mkdir dealer-price-tracker-production-grade && cd dealer-price-tracker-production-grade && git
init
npm init -y          # then add "workspaces": ["server", "web"] to package.json
npm create vite@latest web -- --template react-ts
mkdir -p server/src server/migrations shared
npm i -w server fastify @fastify/cookie @fastify/helmet @fastify/rate-limit @fastify/static pg
zod nodemailer
npm i -D -w server typescript tsx vitest esbuild @types/node @types/pg
npm i -w web @tanstack/react-query react-router-dom
```

- 6.2 Turn on TypeScript `strict` everywhere. Add root scripts: `dev`, `build`, `typecheck`, `test`, `migrate`, `seed:demo`, `smoke`.

Done when: `npm run typecheck` passes on the empty project.

Step 7. Part B · Configuration, database connection and migrations

- 7.1 `server/src/config.ts`: read environment variables with a zod schema (`DATABASE_URL`, `PUBLIC_URL`, `COOKIE_SECURE`, `TRUST_PROXY`, `SESSION_TTL_HOURS`, `RATE_LIMIT_PER_MIN`, `TIMEZONE`, `SMTP_URL`, `SCRAPER_MAX_PAGE_MB`, `SCRAPER_ALLOW_PRIVATE`) and refuse to start with a clear message when one is wrong.
- 7.2 `server/src/db.ts`: a `pg` connection pool with a statement timeout, and a `tx(fn)` helper that runs `fn` inside `BEGIN/COMMIT` and rolls back on any error.
- 7.3 `server/src/migrate.ts`: apply `migrations/*.sql` in name order, each in its own transaction, recorded in a `schema_migrations` table, under a PostgreSQL advisory lock so several app servers can start at the same time.

```
createdb app_dev
export DATABASE_URL=postgres://postgres:postgres@localhost:5432/app_dev
npm run migrate
```

Done when: Running `npm run migrate` twice applies the migrations once and then reports the database is up to date.

Step 8. Part B · Schema and scraper safety

- 8.1 Tables as in the data model; reuse the scraper and history code from Part A unchanged (in `shared/`).
- 8.2 Wrap fetching with a network guard: resolve DNS yourself and refuse loopback, private, link-local and mapped IPv6 answers at connect time (defeats DNS rebinding); re-check every redirect hop; http/https only; max 5 redirects; size cap.

Done when: Tests: private addresses refused directly, via DNS names and via redirects.

Step 9. Part B · Logins, roles and request security

The same proven foundation every production app here uses.

- 9.1 **Passwords:** hash with Node's built-in `scrypt` (memory-hard, no native add-on) and compare in constant time. For an unknown email, verify against a dummy hash so response time doesn't reveal which emails exist. Lock the account for 15 minutes after 5 wrong passwords.
- 9.2 **Sessions:** a random 32-byte token in an `httpOnly`, `SameSite=Lax` cookie (`Secure` in production). Store only the token's SHA-256 with a CSRF token, expiry and last-seen time; expire idle sessions.
- 9.3 **Every request:** a Fastify `onRequest` hook loads the session, refuses writes whose `Origin` is another site, and requires the `x-csrf-token` header on every POST/PUT/DELETE.
- 9.4 **Headers and limits:** Helmet with a strict Content-Security-Policy; `@fastify/rate-limit` keyed by the signed-in session (or IP for visitors), with tighter limits on login.
- 9.5 **Roles:** admin (competitors, scrapes, stock, settings, users) and viewer (dashboard and exports). A `requireUser(...roles)` pre-handler protects each route; the web app hides what a role can't use, but the server is what enforces it.
- 9.6 **Audit log:** an `audit(...)` helper writes who did what, to which record, from which IP, for every business action and every login.
- 9.7 **CLI:** `create-admin` to make the first admin (password from an environment variable or a hidden prompt).

Done when: API tests prove: wrong passwords lock the account, a write without the CSRF token is refused, each role gets 403 on routes it may not use.

Step 10. Part B · Scheduler, runs and digest

- 10.1 Record each run in `scrape_runs`; a partial unique index (`CREATE UNIQUE INDEX scrape_runs_one_running ON scrape_runs ((true)) WHERE status = 'running'`) lets only one run exist at a time across all instances — a second start gets "already running"; runs left "running" by a crashed process are closed as failed.
- 10.2 Daily schedule at the configured time (runs once however many instances check); "Scrape now"; "Test first page" (rate-limited) that shows what the scraper finds without saving.
- 10.3 After each run, rebuild the report and send the digest email (plain text).

Done when: Tests: a second scrape request gets "already running"; the digest is sent after a run.

Step 11. Part B · Stock upload, exports and web app

- 11.1 Stock CSV upload that accepts Excel UTF-8 (byte-order mark, quoted "\$21,500"), refusing files with missing columns, short rows or duplicate stock numbers without changing anything.
- 11.2 CSV exports with formula-injection protection; React pages for dashboard, competitors, scrapes, my stock, settings and users.

Done when: Upload and export tests pass.

Step 12. Part B · Test it end to end

- 12.1 **Unit tests** (Vitest) for every pure business rule in `shared/`.
- 12.2 **API tests** against a real PostgreSQL database: a global set-up drops and recreates the test database (only if its name contains `test`), migrates it and loads the demo data. Tests sign in as each role and call the real HTTP API.
- 12.3 **Browser smoke test** (`web/scripts/smoke.mjs` with Playwright): build, start the real server on a fresh database, then run a real scrape of the simulated sites through the app, add a competitor with Test first page, upload stock and check the dashboard, Fail on any script error, and save screenshots.

```
npm run typecheck && npm test
npm run build && npm run smoke
```

Done when: All three suites pass locally.

Step 13. Part B · Package, automate and document

- 13.1 A multi-stage `Dockerfile`: build everything, then a slim runtime image with production dependencies only, running as a non-root user, with a `HEALTHCHECK` on `/readyz`.
- 13.2 `docker-compose.yml` with PostgreSQL and the app; `.env.example` explaining every setting.
- 13.3 GitHub Actions: typecheck, all tests with a PostgreSQL service container, build, the browser smoke test, a dependency audit, and a job that builds the Docker image and checks it starts and answers.
- 13.4 `README.md` for users and `docs/OPERATIONS.md` for whoever runs it (https, backups, monitoring, upgrades).

Done when: CI is green on GitHub and `docker compose up -d --build` gives a working app.

6. Testing, deployment and known limits

How it is tested

- **Demo:** unit and integration tests (the scraper checked car by car against simulated sites); browser test of the dashboard (desktop, phone, dark mode).
- **Production:** scraping and history across formats; the down day skipped; a failed re-scrape never wipes a good result; one run at a time; daily schedule once; crashed runs closed; digest sent; network-safety refusals (DNS, redirects, size); roles and CSRF; stock upload validation; CSV injection; full-stack browser test.

```
# demo
npm test && npm run build && npm run smoke
# production
npm run typecheck && npm test && npm run build && npm run smoke
```

Deploying and running it

- **Demo:** copy `config/competitors.example.json` to `config/competitors.json`, run `npm run scrape` daily (or let the Daily scrape Action do it) and publish the dashboard statically.

```
cd dealer-price-tracker-production-grade
cp .env.example .env # POSTGRES_PASSWORD, PUBLIC_URL, TIMEZONE, SMTP_URL
docker compose up -d --build
docker compose exec app node dist/cli/create-admin.js --email you@yourdealer.com --name "Your Name"
# then: Competitors → Add → Test first page → Scrape all now; upload My stock
```

Known limits

- JavaScript-only websites need a headless browser (not included).
- Competitors are scraped one after another (50 sites × 20 pages ≈ 50 minutes once a day).
- Prices compared as scraped (same currency assumed).
- Rate limits per server instance.

Sensible next steps

- Headless-browser mode for JavaScript-only sites.
- Feed the scraped market prices into the used-car price estimator as training data.
- Price-change alerts per model you stock.